# Step 1 — A Graph With Only Short-Term (Checkpointer) Memory — Hints

> Back to the step · Hint 1 · Hint 2 · Solution


- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and the one subtlety


## Hint 1 — The idea, and the exact pieces

A normal LangGraph chat graph with only a checkpointer — enough to remember within one conversation, nothing more. Pieces:

- `src/config.py`, `src/exceptions.py`, `src/utils/logger.py`, `src/utils/prompts.py` — the usual.
- `src/models/llm.py` — `get_model` (`ChatOpenAI`).
- `src/state.py` — `ChatState`: `messages: Annotated[list, add_messages]`, `user_id`, `recalled` (the last two matter from Step 2).
- `src/nodes.py` — `chat_node`: fold `state["recalled"]` into the system prompt, call the model, return `{"messages": [reply]}`.
- `src/graph.py` — `build_graph(model, store, checkpointer)`: `START -> chat -> END`, `compile(checkpointer=..., store=...)`.
- `main.py` — build a `SqliteSaver` **inside a `with` block** and loop on a hardcoded `thread_id`.

## Hint 2 — The plan, and the one subtlety

```
state.py:  ChatState(messages[add_messages], user_id, recalled)
nodes.py:  make_chat_node(model) -> chat_node(state):
             SystemMessage(prompt) + messages -> reply
graph.py:  build_graph(model, store, checkpointer):
             START->chat->END; compile(checkpointer, store)
main.py:   with SqliteSaver.from_conn_string("...sqlite") as cp:
             app = build_graph(model, InMemoryStore(), cp)
             loop: app.invoke({messages, user_id, recalled},
                              {configurable:{thread_id}})
```

The subtlety: `SqliteSaver.from_conn_string(...)` is a **context manager** — build the graph and run the loop *inside* the `with` block, or the connection is closed before you use it. Prove the gap by hand: same `thread_id` remembers your name; a new `thread_id` (or a restart with a different id) forgets. That amnesia is what Step 2 fixes. The whole file set is in the Solution.